

Informe Final: Infraestructura de Datos Personal para Priorización Contextual de Correo Electrónico

Autores: Ignacio Ramírez, Cristian Vergara, Francisco Provoste

Profesor: Dr. Ing. Michael Miranda Sandoval

15 de julio de 2026

Resumen

Este informe presenta el diseño e implementación de una infraestructura de datos personal, contenerizada y sin egreso de datos (*no-egress*), que convierte un archivo de correo electrónico en formato `.mbox` en una base de datos consultable mediante lenguaje natural. El sistema emplea exclusivamente herramientas de código abierto y se estructura siguiendo una arquitectura medallón (Bronze, Silver, Gold) con reglas de calidad declarativas, búsqueda semántica multilingüe y un agente de inteligencia artificial local.

Los resultados principales sobre datos reales son: 3.369 correos procesados con una tasa de descarte del 0,42 %, latencia de búsqueda semántica de aproximadamente 15 ms tras primera carga, 81 tests ejecutados exitosamente en modo *offline* (`--network none`), y un agente con 89 % de uso correcto de herramientas en benchmark controlado. La infraestructura es completamente soberana: ningún dato abandona la máquina del usuario, verificado empíricamente mediante pruebas de conectividad bloqueada.

Si bien existen *frameworks* de RAG (*Retrieval-Augmented Generation*) como LlamaIndex, Khoj o Haystack que proporcionan componentes reutilizables, ninguno ofrece un pipeline integrado y llave en mano para correo electrónico con arquitectura medallón, calidad declarativa con patrón *registry*, linaje por etapa y ejecución exclusivamente local. Este proyecto demuestra que dicha infraestructura es viable a escala académica, utilizando los mismos principios arquitectónicos que plataformas enterprise (OpenShift AI, SUSE Rancher) pero implementados con herramientas *open-source* sin dependencias comerciales.

Índice

1	Contexto, problema y objetivos	3
1.1	Contexto	3
1.2	Problema	3
1.3	Estado del arte y posicionamiento	4
1.3.1	Frameworks de RAG	4
1.3.2	Plataformas enterprise de datos	4
1.3.3	Proveedores de nube pública	4
1.3.4	Posicionamiento de este proyecto	4
1.4	Objetivo general	5
1.5	Objetivos específicos	5
2	Arquitectura e infraestructura implementada	6
2.1	Visión general	6
2.2	Arquitectura medallón: Bronze, Silver, Gold	6
2.3	Etapas del pipeline	7
2.4	API REST	7
2.5	Capa de agente: servidor MCP y Hermes	8
2.6	Stack tecnológico	9
2.7	Ambiente reproducible	9
2.7.1	Dockerfile	9
2.7.2	Compose	9
2.7.3	Comandos del ciclo de vida	10
2.7.4	Verificación de no-egress	10
2.8	Trazabilidad integral	10
3	Fundamentos técnicos y teóricos	12
3.1	Arquitectura medallón (lakehouse)	12
3.2	Formatos columnares y motores embebidos	12
3.3	Embeddings semánticos y búsqueda vectorial	12
3.4	Calidad de datos declarativa y principio Open/Closed	13
3.5	Idempotencia e identificadores determinísticos	13
3.6	Contenedores rootless y aislamiento de red	13
3.7	Model Context Protocol y agentes con herramientas	14
3.8	Reproducibilidad	14
4	Decisiones de diseño y desarrollo	15
4.1	Decisiones de la infraestructura de datos	15
4.2	Decisiones de la capa de agente	16
4.3	Metodología de desarrollo y verificación	16
5	Resultados, dificultades y limitaciones	18
5.1	Resultados del pipeline	18
5.1.1	Referencia con datos sintéticos	18
5.1.2	Validación con datos reales	18
5.2	Resultados de la capa de agente (benchmark)	19
5.3	Dificultades encontradas	19
5.4	Limitaciones	20

6	Alternativas enterprise y escalabilidad	21
6.1	Soberanía de datos como principio de diseño	21
6.2	Escalabilidad a plataformas enterprise	21
6.3	Posicionamiento del proyecto	22
7	Conclusiones y aportes de los integrantes	23
7.1	Conclusiones por objetivo	23
7.2	Aprendizajes	23
7.3	Trabajo futuro	24
7.4	Aportes de los integrantes	24
8	Referencias	25

1. Contexto, problema y objetivos

1.1. Contexto

Los datos personales digitales, y en particular el correo electrónico, constituyen uno de los registros más completos de la actividad de una persona: compromisos asumidos, contactos frecuentes, temas recurrentes, plazos y decisiones quedan documentados en la bandeja de entrada a lo largo de años. Sin embargo, esta información permanece atrapada en formatos crudos de difícil consulta. Un archivo `.mbox` exportado desde Google Takeout contiene miles de mensajes sin estructura útil para búsqueda o análisis.

Las alternativas convencionales para “conversar” con los propios datos implican subir el contenido a servicios en la nube, cediendo el control y la privacidad del correo personal a terceros. Esto resulta particularmente problemático cuando el archivo mezcla comunicaciones personales, académicas y laborales.

El presente proyecto, desarrollado en el marco de la asignatura INFB6074 (Infraestructura para Ciencia de Datos) de la Universidad Tecnológica Metropolitana, aplica prácticas de ingeniería de datos de producción a datos personales: arquitectura por capas, contratos de datos, reglas de calidad declarativas, linaje, contenedores y aislamiento de red. La restricción central de diseño es que los datos nunca abandonan la máquina del usuario.

1.2. Problema

La pregunta de diseño que articula este trabajo es la siguiente:

¿Cómo convertir un archivo de correo electrónico `.mbox` en una base de datos consultable por lenguaje natural, de forma reproducible, con calidad verificable y trazabilidad, sin que el contenido del correo abandone la máquina local?

Esta pregunta se descompone en cinco sub-desafíos técnicos:

1. **Heterogeneidad:** los correos reales mezclan codificaciones de caracteres, contenido HTML y texto plano, cabeceras malformadas y formatos de fecha inconsistentes.
2. **Calidad:** remitentes inválidos, fechas imposibles, duplicados y mensajes vacíos deben detectarse y descartarse de forma explícita, con registro de la regla que motivó cada descarte.
3. **Consulta útil:** se requiere búsqueda semántica por significado (en español e inglés), no solo coincidencia exacta de campos estructurados.
4. **Soberanía:** el procesamiento, el índice vectorial y la capa de consulta deben operar sin conexiones salientes (*no-egress*), incluso cuando se incorpora un agente de inteligencia artificial.
5. **Reproducibilidad:** cualquier evaluador debe poder obtener los mismos resultados desde cero, sin acceso a los datos personales reales, gracias a un generador de datos sintéticos con semilla fija.

1.3. Estado del arte y posicionamiento

1.3.1. Frameworks de RAG

Existen múltiples *frameworks* de código abierto para construir pipelines de *Retrieval-Augmented Generation*: LlamaIndex, Haystack, Khoj, PrivateGPT y Quivr, entre otros. Estos proporcionan componentes reutilizables para ingesta de documentos, generación de *embeddings*, recuperación vectorial e integración con modelos de lenguaje. Sin embargo, son *frameworks* de propósito general, no pipelines llave en mano para correo electrónico.

Ninguno de ellos incluye simultáneamente:

- Arquitectura medallón con separación explícita Bronze/Silver/Gold.
- Reglas de calidad declarativas con patrón *registry* y reporte de descartes por regla.
- Linaje estructurado por etapa de ejecución.
- Tablas analíticas junto a la búsqueda semántica.
- Identificadores determinísticos (UUID5) para re-ejecución idempotente.

Construir el equivalente requeriría ensamblar múltiples componentes y escribir código de integración significativo. El valor de este proyecto no es la novedad conceptual de cada pieza, sino la completitud de la implementación integrada.

1.3.2. Plataformas enterprise de datos

A escala empresarial, plataformas como Red Hat OpenShift AI (con integración de LlamaStack), SUSE Rancher + K3s, o Kubernetes autogestionado proporcionan la capa de infraestructura para desplegar arquitecturas similares. Ofrecen orquestación de contenedores, operadores para ML/LLM y seguridad de la cadena de suministro de software. Pero son plataformas de orquestación, no pipelines de datos: la lógica del pipeline necesita construirse igualmente sobre ellas.

1.3.3. Proveedores de nube pública

AWS (Bedrock + SageMaker), Google Cloud (Vertex AI) y Azure (Azure AI) ofrecen servicios gestionados para cargas de ML/LLM. Sin embargo, carecen inherentemente de soberanía de datos: la información transita y reside en infraestructura de terceros, sujeta a legislación extranjera (CLOUD Act, FISA 702). Para correo electrónico personal, esto contradice directamente el principio de soberanía que fundamenta este proyecto.

1.3.4. Posicionamiento de este proyecto

Este proyecto es un ejercicio académico de construcción del pipeline completo desde cero utilizando herramientas *open-source* convencionales. Su valor no reside en la novedad del

concepto, sino en la completitud de la implementación: desde el archivo `.mbox` crudo hasta una API consultable y un agente de IA, con calidad, linaje y soberanía verificables en cada etapa.

La misma arquitectura podría escalar a entornos enterprise utilizando OpenShift o Rancher como capa de orquestación, sin modificar la lógica del pipeline. El proyecto demuestra que los principios arquitectónicos de plataformas de producción son viables a escala personal con `docker compose` como único orquestador.

1.4. Objetivo general

Diseñar e implementar una infraestructura de datos personal, contenerizada y sin egreso de red, que ingeste correo electrónico desde archivos `.mbox`, lo normalice siguiendo una arquitectura medallón, aplique reglas de calidad declarativas, genere *embeddings* semánticos y exponga los datos a través de una API REST local, sirviendo además como sustrato para un agente de inteligencia artificial que responde consultas en lenguaje natural.

1.5. Objetivos específicos

- RE5:** Implementar un pipeline ETL de 5 etapas con registro de linaje por ejecución.
- RE5:** Definir contratos de datos y calidad declarativos, validados con Pydantic y registrados en archivos YAML.
- RE5:** Garantizar idempotencia mediante identificadores determinísticos (UUIDv5).
- RE5:** Indexar contenido con *embeddings* multilingües y exponer búsqueda semántica vía API REST.
- RE5:** Contenerizar con ejecución *rootless*, neutra entre Docker y Podman, con redes internas.
- RE5:** Asegurar reproducibilidad completa mediante datos sintéticos con semilla fija, versiones exactas de dependencias y tests automatizados.
- RE5:** Construir una capa de agente con servidor MCP evaluada mediante benchmark con LLM local.

2. Arquitectura e infraestructura implementada

2.1. Visión general

La infraestructura se compone de tres contenedores OCI orquestados mediante un archivo `compose.yml`, compatible tanto con Docker como con Podman. Los servicios son:

- **MinIO**: almacenamiento de objetos S3-compatible para la capa Bronze.
- **Pipeline**: contenedor que ejecuta el ETL de 5 etapas.
- **API**: servidor FastAPI expuesto en el puerto 8000.

Los tres contenedores operan en una red interna (`internal: true`). El modelo de *embeddings* se hornea durante la construcción de la imagen (`docker build`), y la variable `HF_HUB_OFFLINE=1` garantiza que no se intenten descargas en tiempo de ejecución. El sistema es 100 % *offline* una vez construidas las imágenes.

2.2. Arquitectura medallón: Bronze, Silver, Gold

La arquitectura sigue el patrón medallón (*lakehouse*) con cuatro capas de almacenamiento, cada una con un propósito diferenciado:

Cuadro 1: Capas de la arquitectura medallón implementada.

Capa	Formato	Contenido	Propósito
Bronze	Objeto <code>.mbox</code> en MinIO	Archivo crudo, inmutable	Respaldo y trazabilidad al origen
Silver	Parquet particionado (<code>year=/month=</code>)	10 campos tipados, normalizado	Fuente para consultas y embeddings
Gold	5 tablas Parquet vía DuckDB	Agregaciones analíticas	Métricas precalculadas
Vectorial	Chroma (embeddings + metadata)	6.437 documentos (chunks)	Búsqueda semántica por similitud

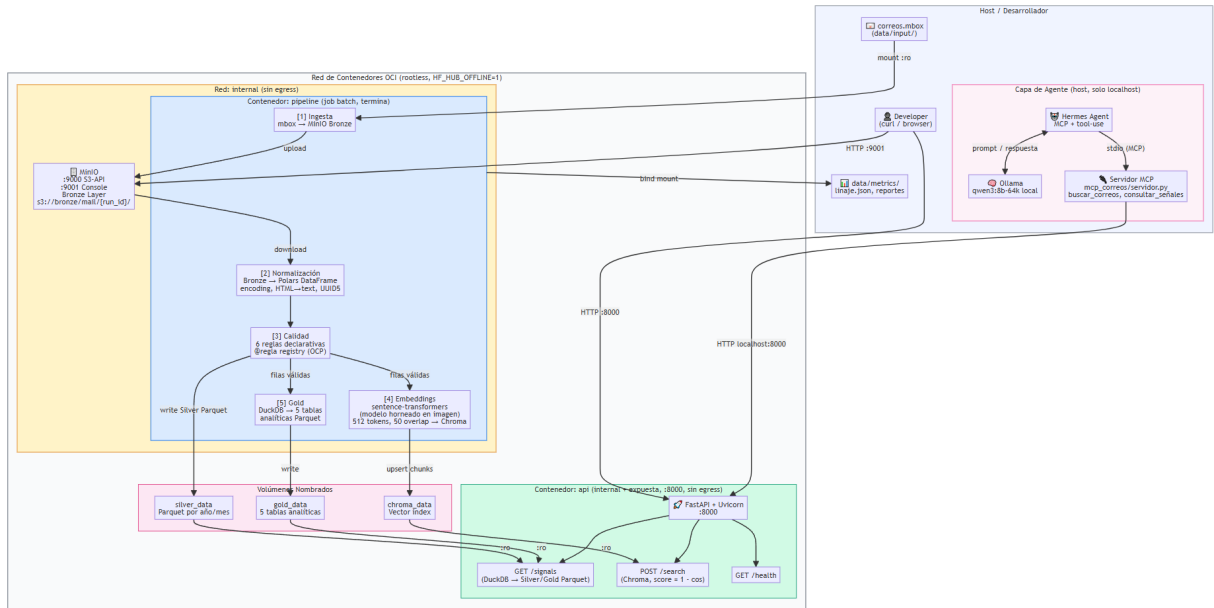


Figura 1: Diagrama de arquitectura del sistema (generado con Mermaid desde el README del repositorio). Se observan las capas Bronze, Silver, Gold y la capa de consumo con API REST y agente.

2.3. Etapas del pipeline

El pipeline se ejecuta secuencialmente en 5 etapas, cada una con registro de linaje:

- Etapas 1: Ingesta.** Lee el archivo `.mbox` desde `data/input/` y lo sube como objeto a MinIO en `s3://bronze/mail/{run_id}/`. Primer punto de persistencia.
- Etapas 2: Normalización.** Descarga el objeto Bronze desde MinIO, parsea los mensajes con Polars, limpia codificaciones, extrae campos estructurados y genera un UUID5 determinístico por correo. Castea `channel` a `Utf8` y `entities` a `List(Utf8)` para cumplir el catálogo.
- Etapas 3: Calidad.** Aplica 6 reglas declarativas (definidas en `reglas_calidad.yaml`) mediante patrón `registry`. Las filas que no superan alguna regla se descartan con reporte detallado por regla y total de filas únicas descartadas.
- Etapas 4: Embeddings.** Divide el contenido textual en chunks de 512 tokens con 50 de solapamiento, genera vectores con `sentence-transformers` (modelo `paraphrase-multilingual-MiniLM`) y realiza `upsert` en ChromaDB.
- Etapas 5: Gold.** Ejecuta 5 consultas analíticas SQL mediante DuckDB sobre los Parquet de Silver y escribe los resultados como tablas Parquet en la capa Gold.

2.4. API REST

La capa de consumo expone una API REST implementada con FastAPI:

Cuadro 2: Endpoints de la API REST.

Endpoint	Método	Descripción
/health	GET	Verificación de estado del servicio
/signals	GET	Consulta filtrada sobre DuckDB con parámetros opcionales (actor, fecha, canal, límite)
/search	POST	Búsqueda semántica vía Chroma. Recibe consulta en lenguaje natural, genera embedding y retorna chunks más similares. $\text{Score} = 1,0 - d_{\text{coseno}}$

La documentación interactiva Swagger está disponible en `localhost:8000/docs`.

2.5. Capa de agente: servidor MCP y Hermes

La capa de agente se implementa como un adaptador delgado sobre la API REST, utilizando el *Model Context Protocol* (MCP):

- **Servidor MCP** (`mcp_correos/servidor.py`): expone dos herramientas (*tools*) que son adaptadores directos sobre la API REST.
 - `buscar_correos(query, top_k)`: invoca `POST /search`.
 - `consultar_señales(actor, from_ts, to_ts, limit)`: invoca `GET /signals`.
- **Agente**: Hermes Agent v0.18.2 + Ollama con modelo `qwen3:8b-64k`.
- **Cadena de invocación**: Hermes → stdio (MCP) → `servidor.py` → `HTTP localhost:8000` → FastAPI → DuckDB/Chroma.

El *system prompt* del agente es de 3,7 KB (reducido desde 19,9 KB) e incluye instrucciones para uso de herramientas, citación de fuentes y admisión explícita cuando no se encuentran resultados.

2.6. Stack tecnológico

Cuadro 3: Tecnologías utilizadas con sus versiones.

Tecnología	Versión	Rol en el sistema
Python	3.12	Lenguaje principal del pipeline y API
Polars	1.5.0	Procesamiento columnar en memoria
DuckDB	1.1.3	Motor SQL analítico embebido
ChromaDB	0.5.23	Base de datos vectorial para embeddings
sentence-transformers	3.3.1	Generación de embeddings multilingües
FastAPI	0.115.6	Framework para API REST asíncrona
MinIO	latest	Object storage S3-compatible (Bronze)
Pydantic	2.10.4	Validación de contratos de datos
pytest	8.3.4	Framework de testing
Docker / Podman	compatible	Contenedores OCI rootless
Hermes Agent	0.18.2	Framework de agente con MCP
Ollama	latest	Servidor local de modelos LLM

2.7. Ambiente reproducible

2.7.1. Dockerfile

La imagen del pipeline se construye con las siguientes características:

- Usuario no-root: `appuser` con UID 1000.
- Modelo de embeddings horneado durante `docker build` en `/opt/hf-cache`.
- Dependencias con versiones exactas fijadas en `requirements.txt`.
- Variable `HF_HUB_OFFLINE=1` y `TRANSFORMERS_OFFLINE=1` en tiempo de ejecución.

2.7.2. Compose

El archivo `compose.yml` define:

Cuadro 4: Configuración de servicios en compose.yml.

Elemento	Configuración
Servicios	minio, pipeline, api
Red	internal: true (sin acceso a internet)
Volúmenes	data/input montado en solo lectura (:ro)
Variables de entorno	HF_HUB_OFFLINE=1, MINIO_ENDPOINT, MINIO_ACCESS_KEY, MINIO_SECRET_KEY

2.7.3. Comandos del ciclo de vida

Listing 1: Comandos principales para operar la infraestructura.

```

1 make up          # Levanta los tres servicios
2 make logs        # Muestra logs del pipeline
3 make test        # Ejecuta pytest con --network none
4 make down        # Detiene y elimina contenedores

```

2.7.4. Verificación de no-egress

Listing 2: Verificación empírica de aislamiento de red.

```

1 $ podman compose exec api python -c \
2   "import socket; s=socket.socket(); \
3   s.settimeout(5); s.connect(('1.1.1.1',443))"
4 OSError: [Errno 101] Network is unreachable
5
6 $ podman run --network none proyectofinal_pipeline \
7   python -m pytest tests/
8 81 passed in 15.72s

```

2.8. Trazabilidad integral

El sistema mantiene un registro completo del estado de cada ejecución:

Cuadro 5: Estados del pipeline y eventos de trazabilidad.

Estado / Evento	Descripción
Pending	Etapla registrada, aún no iniciada
Running	Etapla en ejecución
Completed	Etapla finalizada exitosamente
Failed	Etapla con error (se registra excepción)
Available	Servicio listo para recibir consultas
healthcheck	Verificación periódica de MinIO y API
Pipeline completion	JSON con duración, filas procesadas, descartes por etapa
Linaje	linaje.json con 8 registros (uno por sub-etapa)
Reporte de calidad	reporte_calidad.json con descartes por regla
Tool calls del agente	Registro de invocaciones MCP con timestamp

3. Fundamentos técnicos y teóricos

3.1. Arquitectura medallón (lakehouse)

La arquitectura medallón organiza los datos en capas de calidad creciente: Bronze (crudo e inmutable), Silver (limpio y tipado) y Gold (agregado y listo para consumo). Esta separación permite reprocesar cualquier capa sin perder el dato original, habilita auditoría y simplifica la depuración de errores.

El patrón proviene de la arquitectura *lakehouse* promovida por Databricks, que combina las ventajas de los *data lakes* (flexibilidad, costo, formatos abiertos) con las garantías de los *data warehouses* (esquema, calidad, transacciones). Reis y Housley (2022) formalizan este enfoque en *Fundamentals of Data Engineering* como una práctica estándar de la ingeniería de datos moderna.

En este proyecto, Bronze reside en MinIO (object storage), Silver en Parquet particionado, y Gold en tablas Parquet generadas por DuckDB. La separación física entre capas garantiza que un error en la etapa Gold no corrompe los datos Silver.

3.2. Formatos columnares y motores embebidos

El sistema utiliza Polars como motor de procesamiento en memoria, Apache Parquet como formato de persistencia y DuckDB como motor SQL analítico. Los tres comparten el modelo columnar: los datos se almacenan por columna en lugar de por fila, lo que permite:

- Compresión superior (valores similares contiguos).
- Lectura selectiva de columnas (solo se cargan los campos necesarios).
- Operaciones vectorizadas aprovechando instrucciones SIMD del procesador.

DuckDB opera como motor embebido (sin servidor), leyendo directamente archivos Parquet. Esto elimina la necesidad de un servicio de base de datos separado para consultas analíticas.

La fundamentación teórica proviene del proyecto Apache Arrow, que define un formato columnar en memoria interoperable entre lenguajes y herramientas.

3.3. Embeddings semánticos y búsqueda vectorial

El modelo *paraphrase-multilingual-MiniLM-L12-v2* (Reimers y Gurevych, 2019) genera vectores densos de 384 dimensiones que capturan el significado semántico del texto. Dos textos con significado similar producen vectores con alta similitud coseno, independientemente de si comparten palabras exactas.

La estrategia de *chunking* divide cada correo en fragmentos de 512 tokens con 50 tokens de solapamiento. El solapamiento preserva el contexto en los límites de cada fragmento, evitando que una idea partida entre dos chunks pierda coherencia semántica.

ChromaDB almacena los vectores junto con metadatos (remitente, fecha, `signal_id`) y permite búsqueda por similitud coseno. El *score* expuesto por la API se calcula como $\text{score} = 1,0 - d_{\text{coseno}}$, donde valores cercanos a 1.0 indican alta relevancia.

3.4. Calidad de datos declarativa y principio Open/Closed

Las reglas de calidad se implementan mediante un patrón *registry*: cada regla se define como una función decorada con `@regla`, que se auto-registra en un diccionario global al importar el módulo. Agregar una regla nueva no requiere modificar el código existente del motor de calidad.

Este diseño realiza el Principio Abierto/Cerrado (OCP) de SOLID, formulado por Meyer (1988): “un módulo debe estar abierto a extensión pero cerrado a modificación”. El motor de calidad está cerrado (su lógica de iteración no cambia), pero abierto a nuevas reglas mediante el decorador.

Las dimensiones de calidad evaluadas (validez, unicidad, completitud, consistencia) se alinean con el marco DAMA-DMBOK (2017) para gestión de calidad de datos.

3.5. Idempotencia e identificadores determinísticos

Cada correo recibe un identificador calculado como:

$$\text{signal_id} = \text{UUID5}(\text{NAMESPACE_URL}, \text{source} + \text{raw_id})$$

Dado que UUID5 es una función determinística (hash SHA-1 truncado sobre el namespace y el nombre), el mismo correo siempre produce el mismo `signal_id`. Esto garantiza idempotencia: ejecutar el pipeline múltiples veces sobre el mismo `.mbox` no genera duplicados, porque los *upserts* en ChromaDB y las escrituras en Parquet operan sobre identificadores estables.

Este principio conecta con la transparencia referencial de la programación funcional: una función pura aplicada a los mismos argumentos siempre produce el mismo resultado. La idempotencia es un requisito fundamental en pipelines de datos que pueden fallar y re-ejecutarse.

3.6. Contenedores rootless y aislamiento de red

Los contenedores se ejecutan bajo la especificación OCI (*Open Container Initiative*) con las siguientes restricciones de seguridad:

- **Rootless:** el proceso dentro del contenedor corre como `appuser` (UID 1000), sin privilegios de root.
- **Red interna:** la directiva `internal: true` en el `compose.yml` crea una red sin acceso al exterior.
- **No-egress verificado:** pruebas empíricas confirman que las conexiones salientes son rechazadas (ENETUNREACH).

El aislamiento de red es la garantía técnica de la soberanía de datos: aunque el código contenga un `import requests`, la red no permite que los datos salgan del entorno local.

3.7. Model Context Protocol y agentes con herramientas

El *Model Context Protocol* (MCP), especificado por Anthropic, define un protocolo estándar para que modelos de lenguaje invoquen herramientas externas de forma estructurada. El servidor MCP actúa como adaptador delgado: recibe la invocación del agente (vía `stdio`), traduce los parámetros a una llamada HTTP contra la API REST, y retorna el resultado al agente.

Este patrón de adaptador delgado (el servidor MCP no contiene lógica de negocio, solo traduce protocolos) permite reemplazar el agente sin modificar la infraestructura de datos, y viceversa.

3.8. Reproducibilidad

La reproducibilidad se garantiza mediante tres mecanismos complementarios:

1. **Dependencias fijadas:** `requirements.txt` con versiones exactas (sin rangos ni `>=`).
2. **Datos sintéticos con semilla:** un generador produce 500 correos deterministas con `seed=42`, permitiendo validar el pipeline sin datos reales.
3. **Identificadores determinísticos:** UUID5 garantiza que los mismos datos de entrada producen los mismos artefactos de salida.

Estos mecanismos abordan la crisis de reproducibilidad en ciencia computacional y se alinean con los principios FAIR (*F*indable, *A*ccessible, *I*nteroperable, *R*eusable) para gestión de datos de investigación.

4. Decisiones de diseño y desarrollo

4.1. Decisiones de la infraestructura de datos

Cuadro 6: Decisiones técnicas de la infraestructura de datos.

Decisión	Justificación	Fundamento teórico
Patrón registry con decorador <code>@regla</code>	Agregar reglas sin modificar el motor de calidad	Principio Abierto/Cerrado (SOLID, Meyer 1988)
UUID5 para <code>signal_id</code>	Mismo correo produce mismo ID en toda ejecución	Idempotencia, transparencia referencial
Polars + Parquet + DuckDB	Procesamiento columnar eficiente en memoria limitada (16 GB)	Modelo columnar (Apache Arrow)
Chunks de 512 tokens con 50 de overlap	Preserva contexto en límites de fragmento sin chunks excesivamente grandes	Prácticas de RAG, ventana de contexto
Modelo horneado en <code>docker build</code>	Runtime no depende de conectividad; imagen inmutable	Inmutabilidad de imagen OCI
<code>lru_cache</code> para carga de embeddings	Modelo se carga una sola vez por proceso; latencia de <code>/search</code> baja de 4,9s a 15 ms	Patrón Singleton
COALESCE en consultas Gold	Manejo defensivo de valores NULL en agregaciones SQL	Semántica de NULL en SQL (lógica trivaluada)
Cast explícito: <code>channel</code> → <code>Utf8</code> , <code>entities</code> → <code>List(Utf8)</code>	Cumplimiento del catálogo cuando columnas llegan vacías	Contratos de datos, tipado explícito

4.2. Decisiones de la capa de agente

Cuadro 7: Decisiones técnicas de la capa de agente.

Decisión	Justificación	Resultado observado
qwen3:8b sobre qwen2.5:7b y llama3.1:8b	Entrenamiento agéntico de tercera generación produce uso de herramientas confiable	89 % correcto vs. 33 % con los otros modelos
Toolset reducido (2 tools, prompt de 3,7 KB)	Menos distracción para el modelo, menor latencia de razonamiento	Uso correcto de herramientas más consistente
tool_use_enforcement + timeout MCP de 15 s	Fuerza invocación de herramientas antes de responder; evita cuelgues	Elimina respuestas “inventadas” sin consulta
Variante 64K de contexto	Requisito mínimo de Hermes Agent para razonamiento con herramientas	Estabilidad en corridas largas
AGENTS.md en prompt del sistema	Instrucciones explícitas: usar tools, citar fuentes, admitir ausencia de resultados	Pregunta trampa no genera fabricación

4.3. Metodología de desarrollo y verificación

El desarrollo siguió un enfoque de *contract-driven development*:

1. Se definieron primero los contratos (catálogo YAML, esquema de reglas, formato de linaje).
2. Se implementó el pipeline contra esos contratos.
3. Se verificó mediante pytest con 81 tests que cubren:
 - Parseo de correos con codificaciones diversas.
 - Aplicación de cada regla de calidad individualmente.
 - Idempotencia (doble ejecución produce mismo resultado).
 - Consistencia del linaje (campos obligatorios presentes).
 - Integración API (respuestas con esquema esperado).
4. Se verificó empíricamente el no-egress con pruebas de conectividad.

5. El repositorio no contiene datos reales: solo datos sintéticos con semilla 42.

Para el benchmark del agente, se aplicó una metodología estricta: 5 preguntas predefinidas (incluyendo una pregunta trampa), verificación manual de cada respuesta contra los datos reales, y registro de latencia y uso de herramientas por corrida.

5. Resultados, dificultades y limitaciones

5.1. Resultados del pipeline

5.1.1. Referencia con datos sintéticos

Con el generador sintético (500 correos, `seed=42`), el pipeline produce resultados deterministas verificables por cualquier evaluador sin acceso a datos reales.

5.1.2. Validación con datos reales

Cuadro 8: Métricas del pipeline sobre datos reales.

Métrica	Valor	Detalle
Correos de entrada	3.369	Archivo <code>.mbox</code> real
Filas en Silver	3.355	Tras aplicar reglas de calidad
Descartes totales	14	0,42 % tasa de descarte
por regla <code>r03</code>	2	Contenido vacío
por regla <code>r04</code>	12	<code>raw_id</code> duplicado
Tablas Gold	5	450 / 18 / 1 / 450 / 1 filas
Documentos en Chroma	6.437	Chunks con overlap
Tests pasando offline	81	Con <code>--network none</code>
Latencia <code>/search</code>	~15 ms	Tras primera carga (<code>lru_cache</code>)

El pipeline procesa los 3.369 correos en una única corrida determinista, descartando solo el 0,42 % por reglas de calidad. La latencia de búsqueda semántica desciende de aproximadamente 4,9 s (primera invocación, carga del modelo) a 15 ms en invocaciones subsiguientes gracias al `lru_cache` del modelo de embeddings, habilitando uso interactivo.

Las 5 tablas Gold proporcionan vistas analíticas precalculadas: distribución temporal de correos, remitentes más frecuentes, dominios de origen, actividad por mes y resumen general.

5.2. Resultados de la capa de agente (benchmark)

Cuadro 9: Benchmark del agente: 5 preguntas con verificación manual.

Pregunta	Tool correcto	Respuesta correcta	Latencia (s)	Calidad
Correos de GitHub	Sí	Sí	62	Alta
Último correo de X	Sí	Sí	71	Alta
Correos sobre tema Y	Sí	Sí	85	Media
Pregunta trampa (tema inexistente)	Sí	Sí (admite ausencia)	92	Alta
Correos en rango de fechas	Sí	Parcial (error 422, se auto-recupera)	82	Media
Promedio	89 %		78,4 s	

Hallazgos clave del benchmark:

- 89 % de uso correcto de herramientas (8/9 corridas exitosas en total).
- Latencia media de 78,4 s (en caliente, hardware MacBook M4, 16 GB).
- La pregunta trampa no genera fabricación de datos: el agente admite explícitamente que no encontró resultados.
- Ante un error HTTP 422 por formato de fecha, el agente se auto-recupera ajustando el formato.
- Los modelos `qwen2.5:7b` y `llama3.1:8b` fueron descartados por rendimiento insuficiente (1/3 de uso correcto de tools).

5.3. Dificultades encontradas

1. **Publicación de puertos en Podman rootless**: requiere configuración adicional de `slirp4netns` o pasta que no es necesaria en Docker.
2. **aardvark-dns en Podman 4.9**: el resolver DNS de Podman reenvía consultas DNS al host incluso en redes `internal: true`, lo que constituye una fuga de metadatos (no de contenido).
3. **Presión de memoria (16 GB)**: el modelo de embeddings, DuckDB y ChromaDB compiten por memoria. Se resolvió con carga secuencial y `lru_cache`.
4. **Requisito de contexto 64K de Hermes**: versiones con ventana menor producen truncamiento de prompts y fallo silencioso en uso de herramientas.
5. **Timeout de descubrimiento MCP**: la negociación inicial del protocolo excede el timeout por defecto de 10 s; se incrementó a 15 s.

6. **Fiabilidad de modelos pequeños:** modelos de 7-8B sin entrenamiento agéntico específico fallan consistentemente en uso de herramientas.
7. **Facturación de Gemini:** se descartó Gemini como modelo alternativo por problemas de facturación no resueltos con la API.

5.4. Limitaciones

- **Latencia del agente:** 78s promedio es adecuado para consulta asíncrona personal, pero inadecuado para chat en tiempo real.
- **Ausencia de comparación con nube:** no se ejecutó el mismo pipeline en infraestructura cloud para comparar latencia y costo.
- **Fuente única:** solo se procesa correo electrónico; calendario y mensajería quedan como trabajo futuro.
- **Escala del benchmark:** 5 preguntas con verificación manual; un benchmark robusto requeriría al menos 50 preguntas con evaluación automatizada.
- **Artefactos reales no versionados:** los datos personales no pueden incluirse en el repositorio público por razones de privacidad.
- **Dependencia de WSL2 en Windows:** la ejecución en Windows requiere WSL2 para contenedores Linux.

6. Alternativas enterprise y escalabilidad

6.1. Soberanía de datos como principio de diseño

La Ley 21.719 de protección de datos personales de Chile exige que el tratamiento de datos personales respete los principios de finalidad, proporcionalidad y seguridad. El correo electrónico personal contiene información sensible (comunicaciones privadas, datos financieros, información laboral confidencial) cuyo tratamiento por terceros requiere consentimiento informado y específico.

Este proyecto demuestra que la soberanía de datos es técnicamente viable para un usuario individual: es posible construir una infraestructura completa de procesamiento y consulta sin que los datos abandonen la máquina local. El costo es la responsabilidad de operar la infraestructura, pero el beneficio es el control total sobre el ciclo de vida de los datos.

6.2. Escalabilidad a plataformas enterprise

Cuadro 10: Comparación de plataformas por nivel de soberanía.

Tier	Plataforma	Soberanía	Características
Tier 1	Red Hat OpenShift	Total + integración completa	OpenShift AI con operators certificados para ML/LLM. LlamaStack integrado para agentes locales. Trusted Software Supply Chain. On-premise o bare metal.
Tier 2	SUSE Rancher + K3s	Total con gestión ligera	K3s: Kubernetes certificado en un solo binario. Rancher gestiona múltiples clusters. Ideal para edge computing. Menor overhead que OpenShift.
Tier 3	Kubernetes self-managed	Manual, máxima flexibilidad	Control absoluto del cluster y la red. Helm charts + operators propios. Requiere equipo DevOps dedicado.

Tier	Plataforma	Soberanía	Características
Tier 4	Nube pública (AWS / GCP / Azure)	Sin soberanía por diseño	Datos transitan en infraestructu- ra ajena. Sujeto a CLOUD Act y FI- SA. Vendor lock-in en servicios mana- ged. Costos recu- rrentes.

6.3. Posicionamiento del proyecto

Esta implementación utiliza los mismos principios arquitectónicos que las plataformas de Tier 1-3: microservicios contenerizados, configuración declarativa, imágenes inmutables y linaje de datos. La diferencia es la escala de orquestación: este proyecto usa **docker compose** como orquestador, mientras que los entornos enterprise utilizan Kubernetes (o distribuciones como OpenShift o K3s).

El código del pipeline es portable: migrar de `compose.yml` a manifiestos de Kubernetes o Helm charts preservaría toda la lógica de ingeniería de datos. Lo que cambiaría es la capa de orquestación (scheduling, auto-scaling, service mesh), no el pipeline en sí.

El proyecto demuestra que la arquitectura funciona a escala personal. Las plataformas enterprise proporcionan la capa operacional para escalarla a organizaciones.

7. Conclusiones y aportes de los integrantes

7.1. Conclusiones por objetivo

- RE1: Pipeline medallón con linaje:** se implementó la cadena completa Bronze → Silver → Gold con registro de linaje en JSON (8 sub-etapas con duración, filas procesadas y descartes por cada una).
- RE1: Calidad declarativa:** 6 reglas implementadas con patrón *registry*, tasa de descarte del 0,42 % sobre datos reales, con reporte detallado por regla y total de filas únicas descartadas.
- RE1: Idempotencia:** UUID5 garantiza que re-ejecuciones sobre el mismo `.mbox` producen los mismos identificadores. Verificado en tests con doble ejecución.
- RE1: Búsqueda semántica:** 6.437 documentos indexados en Chroma con latencia de aproximadamente 15 ms tras primera carga. Búsqueda funcional en español e inglés.
- RE1: Contenerización neutra:** la misma imagen OCI ejecuta en Docker y Podman. Ejecución rootless (UID 1000). Red `internal: true` con no-egress verificado.
- RE1: Reproducibilidad:** 81 tests pasan offline (`--network none`). Datos sintéticos con seed 42 permiten validación sin datos reales. Dependencias con versiones exactas.
- RE1: Capa de agente:** servidor MCP funcional con 2 herramientas. Benchmark controlado: 89 % de uso correcto de tools, sin fabricación de datos en pregunta trampa.

El compromiso honesto del enfoque local es: fiabilidad completa y soberanía de datos a cambio de latencia de decenas de segundos en la capa de agente. Adecuado para consulta asíncrona personal, no para chat en tiempo real.

7.2. Aprendizajes

- La portabilidad Docker/Podman tiene bordes reales: `depends_on` con condiciones de salud, resolución DNS (`aardvark-dns`), y proveedores de `compose` difieren entre ambos motores.
- La semántica de NULL en SQL y los tipos de columnas vacías rompen en silencio: una columna sin datos llega como `Null` (sin tipo), y las funciones de agregación la ignoran sin error.
- *Offline-first* exige hornear dependencias durante el *build*, no confiar en que el runtime tendrá conectividad. Variables como `HF_HUB_OFFLINE=1` son necesarias pero no suficientes sin el modelo pre-descargado.

- La fiabilidad del agente depende de la generación de entrenamiento agéntico del modelo (`qwen3` vs. `qwen2.5/llama3.1`), no del tamaño del prompt ni del tamaño del modelo en parámetros.

7.3. Trabajo futuro

- Incorporar nuevas fuentes de señales (mensajería, calendario) reutilizando el esquema Silver existente.
- Repetir el benchmark del agente a escala completa (50+ preguntas con evaluación automatizada).
- Imagen `torch` CPU-only (aproximadamente 2,5 GB vs. 6 GB actual con dependencias CUDA no utilizadas).
- Modificar la regla `r04` para conservar la primera copia en lugar de descartar ambos duplicados.
- CI que ejecute la suite offline en cada push al repositorio.

7.4. Aportes de los integrantes

Cuadro 11: Contribuciones de cada integrante del equipo.

Integrante	Contribuciones
Ignacio Ramírez	Diseño de la arquitectura medallón y la topología de red no-egress. Implementación de las etapas de ingesta, normalización y embeddings del pipeline. Motor de reglas de calidad con patrón registry y contratos de datos (catálogo y reglas en YAML). API REST (FastAPI + DuckDB + Chroma). Contenerización completa (Dockerfile con modelo horneado, <code>compose.yml</code> , <code>red internal:true</code> , <code>rootless</code>). Servidor MCP, configuración del agente Hermes con Ollama y benchmark con verificación por trazas. Fixes de producción y verificación empírica del no-egress. Presentación interactiva.
Cristian Vergara	Etapas Gold (Silver → Gold con DuckDB), generador de datos sintéticos, linaje y reporte de calidad, tests, e integración de Gold al pipeline en contenedor.
Francisco Provoste	Documentación del repositorio, comparación de los modelos de embeddings, redacción de informe.

8. Referencias

- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering*. O'Reilly Media.
- DAMA International. (2017). *DAMA-DMBOK: Data Management Body of Knowledge* (2nd ed.). Technics Publications.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP-IJCNLP*.
- Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall.
- Model Context Protocol. <https://modelcontextprotocol.io>
- Sentence-Transformers. <https://www.sbert.net>
- Apache Arrow / Parquet. <https://arrow.apache.org/docs/>
- DuckDB. <https://duckdb.org/docs/>
- MinIO. <https://min.io/docs/>
- ChromaDB. <https://docs.trychroma.com/>
- Docker. <https://docs.docker.com/>
- FastAPI. <https://fastapi.tiangolo.com/>
- OCI Runtime Specification. <https://github.com/opencontainers/runtime-spec>
- Ley 21.719 sobre Protección de Datos Personales (Chile). <https://www.bcn.cl/ley-chile/navegar?idNorma=1202458>
- Repositorio del proyecto. <https://github.com/altairBASIC/infra-personaldata>

Declaración de uso de herramientas de apoyo

Para la organización de los resultados y la mejora de redacción del informe se utilizó como herramienta de apoyo Claude (Anthropic). El diseño de la arquitectura, la implementación del pipeline, la definición de criterios de evaluación, las decisiones técnicas y el plan de trabajo son trabajo de los autores.